INTERNATIONAL TRANSACTIONS IN OPERATIONAL RESEARCH

Intl. Trans. in Op. Res. XX (20XX) 1–11
DOI: xx.xxxx/itor.xxxxx

# The one-dimensional cutting stock problem with sequence-dependent setups

Eduardo M. Silva [a,b], Antônio A. Chaves[b], Mauricio G. C. Resende[b] and Silvio A. de Araujo[c]

[a]*Institute of Mathematics and Statistics, UFU, Minas Gerais, Brazil*
[b]*Institute of Science and Technology, UNIFESP, São Paulo, Brazil*
[c]*Institute of Biosciences, Letters and Exact Sciences, UNESP, São Paulo, Brazil*
*E-mail: machado.silva@ufu.br; antonio.chaves@unifesp.br; mauricio.resende@gp.ita.br; silvio.araujo@unesp.br*

**Abstract**

This document provides supplementary information related to the manuscript. In particular, in Section 1, it provides an illustration of sequencing operations for cutting patterns, in Section 2, an illustrative example of *RKO*, along with the corresponding pseudocode, in Section 3, a comparison of *RKO* and *CGH* with higher capacity gaps, and in Section 4, a detailed gap comparison between *RKO* and *CGH* for higher capacity values, an analysis of the instance space.

## 1. Problem description (sequence-dependent setups)

Figure 1 corresponds to a sequencing operation example where three item types can be cut from the stock object: item type 1 (smallest), item type 2, and item type 3 (biggest). Three cutting patterns are presented in the figure, $A_1 = (0, 0, 3)$, $A_2 = (1, 4, 0)$, and $A_3 = (2, 2, 1)$. The order which these cutting patterns are cut is $A_1 \rightarrow A_2 \rightarrow A_3$, totalizing two sequence-setups operations ($\overrightarrow{12}$ and $\overrightarrow{23}$). In order to cut $A_2$ after $A_1$, it is necessary to remove three knives ($m_{21}$, $m_{31}$, and $m_{41}$) used to cut three units of item type 3, and to add one knife to cut one unit of item type 1 ($m_{12}$) and four knives to cut four units of item type 2 ($m_{22}$, $m_{32}$, $m_{42}$, and $m_{52}$), totaling eight sequence-dependent setups (three removals and five insertions). Regarding operation $\overrightarrow{23}$, one knife had to be added to cut an extra unit of item type 1 ($m_{13}$), two knives were removed to cut only two units of item type 2 ($m_{42}$ and $m_{52}$), and one knife was inserted to cut one unit of item type 3 ($m_{23}$), totaling four sequence-dependent setups (two removals and two insertions).

*Author to whom all correspondence should be addressed (e-mail: machado.silva@ufu.br).

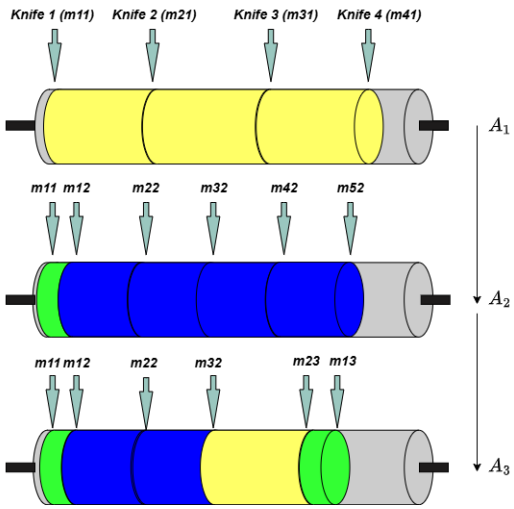



Fig. 1: Sequencing operation example.

## 2. Decoder's pseudo-codes and illustrative example

Algorithm 1 presents the proposed decoder pseudo-code. The algorithm begins by initializing with zero values the parameters used for cutting-pattern construction, namely $M'$ and the vector $\phi$. The residual demands are then set to the initial demands, and the random-key at position $2n + 2$ is used to compute the sorting vector $R$. A for-loop for $j \in [0, \ldots, n + 1]$ is executed to construct each cutting pattern, determine its frequency, and update the residual demand vector accordingly. When this loop ends, a feasible solution with $|P|$ cutting patterns is constructed. The sequence-dependent setups, as well the number of knives are inserted and removed are calculated to obtain the individual fitness. The 2-opt based heuristic (LS-1) is applied to refine the cutting pattern sequencing, and then the KOMBI based heuristic (LS-2) is applied in an attempt to reduce the number of cutting patterns. If KOMBI reduces the number of cutting patterns, then LS-1 is reapplied to improve the new cutting pattern sequence. The individual fitness is then considered as the lowest between fitness when applying LS-1 and LS-2. Finally, the capacity of the solution is calculated and if its value violates the capacity constraints, then a penalty is added to the individual fitness.

Algorithm 2 is designed to minimize sequence-dependent setup costs by optimizing the processing order of the generated cutting patterns. It operates by systematically evaluating pairs of non-adjacent transitions within the current sequence; if breaking these edges and inverting the enclosed sub-route strictly decreases the total number of knife changes, the modified sequence is accepted. To guarantee computational efficiency and prevent cycling, the search explores the neighborhood space until a local optimum is reached or a predetermined threshold of 200 iterations is exhausted, effectively grouping similar patterns together to eliminate the most expensive setup transitions.

Algorithm 3 aims to minimize the total number of active patterns in the solution while strictly enforcing a no-overproduction policy. The algorithm attempts to consolidate existing patterns through a hierarchical reduction strategy, prioritizing a 3-to-2 combination, in which the exact production of three

selected patterns is merged into two newly generated feasible ones, and reverting to a 2-to-1 combination as a fallback. By preserving the exact cumulative production of the original subsets, the method safely reduces setup overhead. A restart mechanism is employed, resetting the failure counter after any successful combination and terminating only after 50 consecutive unsuccessful attempts.

For illustrative purposes, consider the following *1D-CSP* instance: an object of length $L = 5180$; items with lengths $l_1 = 1020$, $l_2 = 1000$, $l_3 = 660$, and $l_4 = 650$; and corresponding demands $d_1 = 18$, $d_2 = 105$, $d_3 = 47$, and $d_4 = 13$. Since $|I| = 4$, each individual is represented by a vector of 11 random keys. Suppose, for instance, the following vector $\chi$:

$$\chi = (.05\ .04\ .57\ .95\ .72\ .69\ .50\ .14\ .80\ .38\ .29).$$

Considering Algorithm 1, we start at line 5 by checking $\chi(10)$ to determine the sorting criterion of this individual. As $\chi(10) = 0.29 < 1/3$, criterion **R3** is selected. To obtain the ordering, we consider the random keys from positions $n+2$ to $2n+1$ of $\chi$. The resulting sub vector has size $n$, where each coordinate index is associated with an item of type $i$. For $\chi$, we have the vector $(\chi(7), \chi(8), \chi(9), \chi(10)) = (0.50, 0.14, 0.80, 0.38)$, where $\chi(7)$ is associated with item $l_1$, $\chi(8)$ with item $l_2$, $\chi(9)$ with item $l_3$, and $\chi(10)$ with item $l_4$. We sort this sub vector in decreasing order of the random key values. The resulting vector is $(\chi(9), \chi(7), \chi(10), \chi(8))$, which implies that the corresponding order is $(l_3, l_1, l_4, l_2)$.

1. We start the loop at line 6 with $j = 0$:
   - Initially, we have $M' = 128.1$ and $\chi(1) = 0.05$, thus by line 9 $\phi(0) = \lceil 2 \cdot 128.1 \cdot 0.05 \rceil = 13$. Then, we enter at the for loop from lines 14-21 with $L' = 0$.
   - Starting with $l_3$, we observe from line 17 that $a_{30} = 3$, since $3 \cdot \phi_1 = 39$, whereas for $a_{30} = 4$, the product $4 \cdot \phi(0)$ exceeds the demand $rd(3) = 47$ and $3l_3 \leq L$. We then move to item $l_1$ with $L^{'} = 3l_3 = 1980$.
   - Now, line 17 indicates that $a_{10} = \min\{\lfloor \frac{L-L'}{l_1} \rfloor, 1\}$ as $2 \cdot \phi(0) = 26 > rd(1)$. Therefore $a_{10} = 1$ and $L' = 1980 + 1020 = 3000$.
   - Proceeding to item $l_4$, we have that $a_{40} = \min\{\lfloor \frac{L-L'}{l_4} \rfloor, 1\}$ as $1 \cdot \phi(0) = 13 = rd(4)$. Therefore, $a_{40} = 1$ and $L' = 3650$.
   - The next item in the list is $l_2$ with $a_{20} = \min\{\lfloor \frac{L-L'}{l_2} \rfloor, 8\} = \min\{1, 8\} = 1$ as item type $l_4$ fits only one time, since $L - L' = 1530$.
   - As the sorting list is empty, the construction of cutting pattern 0 is completed, yielding $a_0^T = (1\ 1\ 3\ 1)$ with a waste of 530. The cutting pattern frequency is computed according to line 22.

$$x_0 = \left\lfloor \min_i \left\{ \frac{18}{1}, \frac{105}{1}, \frac{47}{3}, \frac{13}{1} \right\} \right\rfloor = 13.$$

   - The residual demands are updated according to line 24 as follows: $rd(1) = 18 - 13 = 5, rd(2) = 105 - 13 = 92, rd(3) = 47 - 39 = 8$, and $rd(4) = 13 - 13 = 0$.
2. Back to line 6 we take $j = 1$:
   - $M' = 0.7 \cdot (5 + 93 + 8) = 74.2$
   - As $\chi(1) = 0.04$ line 9 implies $\phi(1) = \lceil 2 \cdot 74.2 \cdot 0.04 \rceil = 6$. Then, we enter at the for loop from lines 14-21 with $L' = 0$.

- Starting with $l_3$, we observe from line 17 that $a_{31} = \min\{\lfloor \frac{L-L'}{l_3} \rfloor, 1\}$ as $2 \cdot \phi(1) = 12 > rd(3)$. Therefore, $a_{31} = 1$. $L' = 660$
- Moving to item type 1 line 17 indicates that $a_{11} = \min\{\lfloor \frac{L-L'}{l_1} \rfloor, 0\}$ as $1 \cdot \phi(0) = 6 > rd(1)$. Therefore, item type 1 is not cut from cutting pattern 1, i.e., $a_{11} = 0$.
- Now, as $rd(4) = 0$, item type 4 does not satisfies the if condition at line 15, hence $a_{40} = 0$.
- The next item in the list is $l_2$ with $a_{22} = \min\{\lfloor \frac{L-L'}{l_2} \rfloor, 15\} = \min\{4, 8\} = 4$ as item type $l_4$ fits only four times, since $L - L' = 4520$.
- As the sorting list is empty, the construction of cutting pattern 1 is completed, yielding $a_1^T = (0\ 4\ 1\ 0)$ with a waste of 520. The cutting pattern frequency is computed according to line 22.

$$x_1 = \left\lfloor \min_i \left\{ \frac{92}{4}, \frac{8}{1} \right\} \right\rfloor = 8.$$

- The residual demands are updated according to line 24 as follows: $rd(2) = 92 - 32 = 60$ and $rd(3) = 8 - 8 = 0$.

3. Back to line 6 we take $j = 2$:
   - $M' = 0.7 \cdot (5 + 61) = 46.2$
   - As $\chi(2) = 0.57$ line 11 implies $\phi(2) = 1$. Then, we enter at the for loop from lines 14-21 with $L' = 0$.
   - Starting with $l_3$, as $rd(3) = 0$, condition at line 15 implies that $a_{32} = 0$.
   - Moving to item type 1 line 17 indicates that $a_{12} = \min\{\lfloor \frac{5180}{l_1} \rfloor, 5\} = 5$ as $5 \cdot \phi(0) = 5 = rd(1)$. Therefore, as item type 1 fits at most 5 times considering the current value of $L'$, we have that $a_{12} = 5$. $L' = 5100$
   - Now, as $rd(4) = 0$, item type 4 does not satisfies the if condition at line 15, hence $a_{42} = 0$.
   - The next item in the list is $l_2$ which does not satisfies condition in line 17 as $L - L' = 80$.
   - As the sorting list is empty, the construction of cutting pattern 2 is completed, yielding $a_2^T = (5\ 0\ 0\ 0)$ with a waste of 80. The cutting pattern frequency is computed according to line 22.

$$x_2 = \left\lfloor \min_i \left\{ \frac{5}{5} \right\} \right\rfloor = 1.$$

   - The residual demands are updated according to line 24 as follows: $rd(1) = 5 - 5 = 0$.

4. Back to line 6 we take $j = 3$:
   - $M' = 0.7 \cdot (60) = 42$
   - As $\chi(3) = 0.57$ line 11 implies $\phi(3) = 1$. Then, we enter at the for loop from lines 14-21 with $L' = 0$.
   - Starting with $l_3$, as $rd(3) = 0$, condition at line 15 implies that $a_{33} = 0$.
   - Moving to item type 1, as $rd(1) = 0$, condition at line 15 implies that $a_{13} = 0$.
   - Now, as $rd(4) = 0$, item type 4 does not satisfies the if condition at line 15, hence $a_{43} = 0$.
   - The next item in the list is item type 2 and line 17 indicates that $a_{23} = \min\{\lfloor \frac{5180}{l_2} \rfloor, 93\} = 5$. Therefore, $a_{23} = 5$ and $L' = 5000$
   - As the sorting list is empty, the construction of cutting pattern 3 is completed, yielding $a_3^T = (0\ 5\ 0\ 0)$ with a waste of 180. The cutting pattern frequency is computed according to line 22.

$$x_3 = \left\lfloor \min_i \left\{ \frac{60}{5} \right\} \right\rfloor = 12.$$

- The residual demands are updated according to line 24 as follows: $rd(2) = 60 - 60 = 0$.

5. As $rd(i) = 0$ for all item type $i$, no more cutting patterns would be generated. The cutting pattern construction is concluded. The obtained solution is:

$$a_0 = \begin{pmatrix} 1 \\ 1 \\ 3 \\ 1 \end{pmatrix} \longrightarrow a_1 = \begin{pmatrix} 0 \\ 4 \\ 1 \\ 0 \end{pmatrix} \longrightarrow a_2 = \begin{pmatrix} 5 \\ 0 \\ 0 \\ 0 \end{pmatrix} \longrightarrow a_3 = \begin{pmatrix} 0 \\ 5 \\ 0 \\ 0 \end{pmatrix}$$

6. We now consider lines 28-29 to compute de sequence-dependent setups. $w_0 = 1 + 1 + 3 + 1 = 6$, $w_1 = abs(0-1) + abs(4-1) + abs(1-3) + abs(0-1) = 7$, $w_2 = abs(5-0) + abs(0-4) + abs(0-1) + abs(0-0) = 10$, $w_3 = abs(0-5) + abs(5-0) + abs(0-0) + abs(0-0) = 10$, $w_4 = 5$. The number of knives changes in this solutions is 38.
7. Applying LS-1 the following sequence could be found within 200 iterations. To calculate the total cost, the machine's initial and final empty states (adding and removing all knives) are modeled as a dummy node, $E$.
   **Initial Sequence:** $E \longrightarrow a_0 \longrightarrow a_1 \longrightarrow a_2 \longrightarrow a_3 \longrightarrow E$
   **Cost:** $6 + 7 + 10 + 10 + 5 = 38$ knife changes.

   - **Iteration 1:**
     - Pick $(a_2 \rightarrow a_3)$ transition.
     - Edges Broken: $(a_0 \rightarrow a_1)$ and $(a_2 \rightarrow a_3)$.
     - Inversion:The sub-route $[a_1 \longrightarrow a_2]$ is flipped to $[a_2 \longrightarrow a_1]$.
     - New Sequence: $E \longrightarrow a_0 \longrightarrow a_2 \longrightarrow a_1 \longrightarrow a_3 \longrightarrow E$
     - New Cost: $6 + 9 + 10 + 2 + 5 = 32$ knife changes.
   - **Iteration 2:**
     - Pick $(a_2 \rightarrow a_1)$ transition.
     - Edges Broken: $(E \rightarrow a_0)$ and $(a_2 \rightarrow a_1)$.
     - Inversion: The sub-route $[a_0 \longrightarrow a_2]$ is flipped to $[a_2 \longrightarrow a_0]$.
     - New Sequence: $E \longrightarrow a_2 \longrightarrow a_0 \longrightarrow a_1 \longrightarrow a_3 \longrightarrow E$
     - New Cost: $5 + 9 + 7 + 2 + 5 = 28$ knife changes.

   By performing these two inversions, the local search successfully eliminates the worst transitions and groups the most similar patterns together reaching the optimal global sequence:

$$a_2 = \begin{pmatrix} 5 \\ 0 \\ 0 \\ 0 \end{pmatrix} \longrightarrow a_0 = \begin{pmatrix} 1 \\ 1 \\ 3 \\ 1 \end{pmatrix} \longrightarrow a_1 = \begin{pmatrix} 0 \\ 4 \\ 1 \\ 0 \end{pmatrix} \longrightarrow a_3 = \begin{pmatrix} 0 \\ 5 \\ 0 \\ 0 \end{pmatrix}$$

The total number of knife changes considering this sequencing is 28.

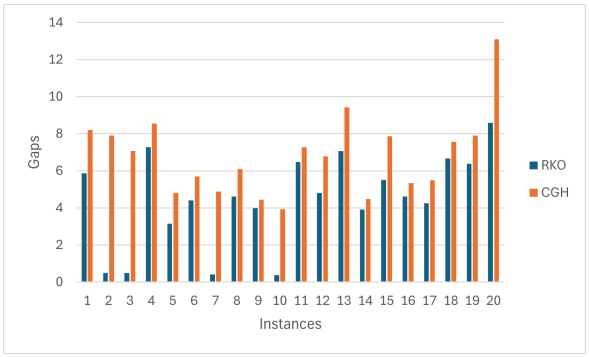
8. Now, when applying LS-2, all attempted combinations for 3–2 and 2–1 reductions were rejected due to the demand satisfaction constraints. Therefore, LS-2 was unsuccessful.
9. We then proceed to line 36 to check capacity feasibility of this solution based on the solution sequence and the pattern frequencies ($O = 34$, $SDS = 28$, and $S = 4$ patterns), the capacity evaluation is performed as follows:
   - Total Material Needed is Calculated as $18{\cdot}(1020){+}105{\cdot}(1000){+}47{\cdot}(660){+}13{\cdot}(650) = 162.830$.
   - Parameter $|O|$: Given $L = 5180$, the parameter is $|O| = \lceil (1.8{\cdot}162.830)/5180 \rceil = \lceil 56.58 \rceil = 57$.
   - Capacity Used: Considering $O_{time} = 1$ and $K_{time} = 2$, the total capacity consumed by the solution is $(1 \times 34) + (2 \times 28) = 34 + 56 = 90$.
   - Capacity limit: with $|P| = 4$ item types, the limit is defined as $|O| + (C \cdot |P|) = 57 + 4 \times C$.
   - Feasibility Status (line 37) The solution is feasible (non-violated) translates to $90 \leq 57 + 4 \cdot C$. Therefore, the current sequencing is strictly feasible as long as the predefined $C \geq 8.25$.
10. $fitness(D(\chi))$ : Assuming the feasibility condition holds, the objective value is calculated as $5180 \cdot (34) + 4 + 518 \cdot (28) = 176.120 + 4 + 14.504 = \mathbf{190.628}$. ( If the capacity is violated, a penalty of $10.733,050$ would be added to this value).

## 3. Higher capacity gaps comparison for *RKO* and *CGH* in classes 11,12, 17 and 18

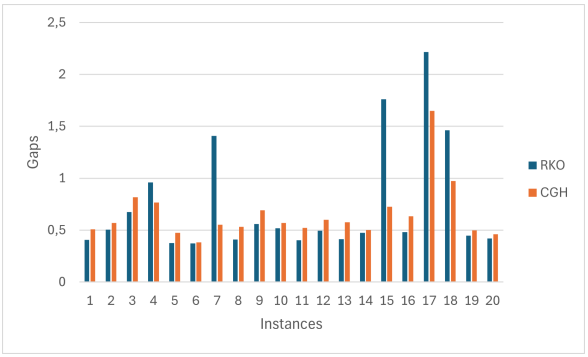
Figure 2 presents the gaps for *RKO* and *CGH* considering the solutions from classes 11, 12, 17, and 18. Figure 2a presents the results for class 11. The *RKO* achieves the smallest gaps for all instances. However, it attains a gap below $1\%$ in only four instances: 2, 3, 7, and 10. Figure 2b presents the gaps for class 12. The gaps for both methods remain similar, however, *RKO* performs worse in five instances, namely instances 4, 7, 15, 17, and 18. Figure 2c presents the gaps plots for class 17. The gaps remain similar for some instances, where *RKO* did not yields smaller values only for instance 2. Finally, Figure 2d presents the gap results for class 18. The gaps for *RKO* and *CGH* remain similar for most instances, with *RKO* being outperformed only in instances 3, 5, 6, 13, and 16.

## 4. Instance Space analysis (ISA)

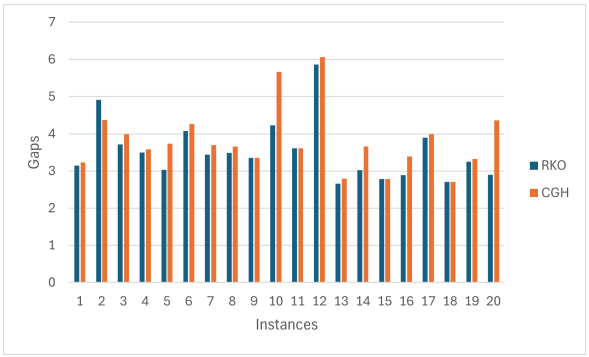
The ISA projection is defined by two macro-statistical dimensions. The X-axis captures the overall magnitude of the instance, increasing as both item lengths and demand volumes grow. The Y-axis acts as a contrast metric, measuring the opposing forces of item length (which pushes values in the positive direction) and demand volume (which pulls values in the negative direction). Analyzing Figure 3 alongside the quadrant distribution, it becomes clear that cutting pattern reductions are remarkably pronounced along the negative spectrum of the Y-axis. The bottom-left quadrant, exclusively populated by Class 1 instances, represents scenarios where the items are so small that, despite a low overall demand, they drive both coordinates to negative values. Furthermore, these small item lengths provide ample packing flexibility, significantly reducing the required number of distinct cutting patterns. Moving to the bottom-right quadrant, the space is dominated by Classes 8 and 14. In this region, the high demand volume overcomes the weight of the item sizes, pulling instances into the negative Y half-space. Here, Class 8



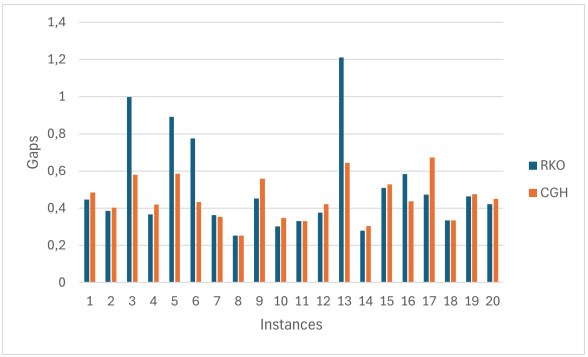

(a) Gaps for class 11.




(b) Gaps for class 12.




(c) Gaps for class 17.




(d) Gaps for class 18.

Fig. 2: Gaps for *RKO* and *CGH* considering classes with medium and high items lengths.

instances demonstrate considerable combinatorial flexibility; their high demand, combined with heterogeneous item lengths, creates highly repeatable configurations that drive substantial setup reductions. Conversely, Class 14 instances in this same quadrant represent a boundary condition where the physical constraints of large items limit further optimization. In contrast, the top-left quadrant is dominated by Classes 7 and 13. In this region, the large dimensions of the items outweigh the low demand, pushing the Y-values into the positive spectrum. This dominance of item size heavily restricts the algorithm's combinatorial space (especially for Class 13) generally yielding marginal or null reductions in cutting patterns. However, Class 7 instances exhibit a degree of combinatorial flexibility due to their wide range of item lengths, which includes smaller sizes, leading to higher pattern reductions for certain instances.

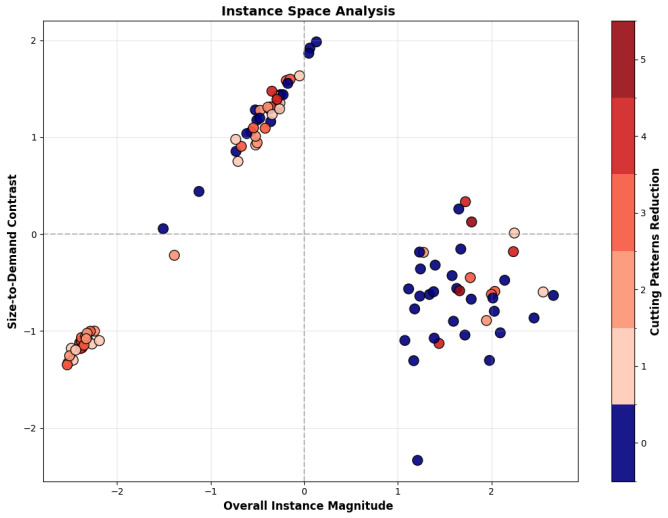



Fig. 3: Instance space analysis of classes 1, 7, 8, 13 and 14 with respect to the minimization of cutting patterns.

**Algorithm 1** Decoder ($D$).

**Require:** $P$,$l_i, L, d_i, n, \chi$
**Ensure:** solution fitness of $D(\chi)$
Define $M' = 0$
$J = \emptyset$
$\phi(j) = 0$ for $j \in [0, ..., n+1]$
$rd(i) = d_i$ for $i \in [0, ..., |I| - 1]$
Check the value of $\chi$(2n+2) to determine $R$ which is a $|I|$-dimensional vector consisting of a permutation of the index set $I$ according to rules R1, R2 or R3
**for** $j \in [0, ..., n+1]$ **do** ▷ each $j$ represents a possible cutting pattern
  $M' = 0.7 \sum_{i \in P} rd(i)$
  **if** $\chi(j) \leq 0.5$ **then**
    $\phi(j) = \lceil 2\chi(j) \cdot M' \rceil$
  **else**
    $\phi(j) = 1$ ▷ considers less heterogeneous cutting patterns
  **end if**
  $L' = 0$
  **for** $i \in R$ **do** ▷ Start cutting pattern $(a_{1j}, ...a_{|I|j})$ construction
    **if** $rd(i) > 0$ and $L' + l_i \leq L$ **then**
      $a_{ij} = 1$
      **while** $L' + (a_{ij} + 1)l_i \leq L$ and $\phi(j)a_{ij} \leq rd(i)$ **do**
        $a_{ij} = a_{ij} + 1$
      **end while**
    **end if**
  **end for** ▷ cutting pattern $j$ is obtained
  $x_j = \left\lfloor \min_i \left\{ \frac{rd(i)}{a_{ij}} \;\middle|\; a_{ij} > 0 \right\} \right\rfloor$ ▷ calculate cutting pattern frequency
  **for** $i \in P$ **do**
    $rd(i) = rd(i) - x_j \cdot a_{ij}$ ▷ update items residual demands
  **end for**
  $P = P \cup \{j\}$ ▷ add index $j$ to cutting patterns set $P$
**end for** ▷ Cutting plan with $|P|$ cutting patterns is obtained
$w_0 = \sum_{i \in I} a_{i0}$ and $w_{|J|+1} = \sum_{i \in P} a_{i|J|}$ ▷ compute setups to insert knives to cut first pattern and remove knives of last pattern, respectively
$w_j = \sum_{i \in I} \text{abs}(a_{ij} - a_{ij-1})$ for $j \in J - \{0\}$ ▷ calculate sequence-dependent setups
*fitness*$(D(\chi))$= $\sum_{j \in P}(c_s w_j + c x_j + 1) + c_s w_{|J|+1}$
Apply LS-1 in $D(\chi)$
Aplly LS-2 in $D(\chi)$
**if** LS-2 was successful **then**
  Apply LS-1 in $D(\chi)$
**end if**
$Cap(D(\chi)) = \sum_{j \in P}(K_{time} w_j + O_{time} x_j) + K_{time} w_{|P|+1}$ ▷ Compute capacity consumed by solution $D(\chi)$
**if** $Cap(D(\chi)) > Cap$ **then** ▷ if constraint (**??**) is violated
  *fitness*$(D(\chi))$= *fitness*$(D(\chi))$+ $c_s c|I| + Cap(D(\chi))$ ▷ penalizes individual fitness
**end if**
  **return** fitness$(D(\chi))$

**Algorithm 2** LS-1 (2-opt)

**Require:** Distance matrix $M$ (knife changes), initial sequence $C$ starting and ending at $E$
**Ensure:** Optimized sequence $C$ with minimal total setup cost

$Iter \leftarrow 0$
$Improved \leftarrow$ True
**while** $Improved$ **and** $Iter < 200$ **do**
    $Improved \leftarrow$ False
    $Iter \leftarrow Iter + 1$
    **for each** pair of transitions $(C_{u-1} \rightarrow C_u)$ and $(C_v \rightarrow C_{v+1})$ **do**
        ▷ Check if breaking these edges and inverting the sub-route decreases cost
        $CurrentCost \leftarrow M_{C_{u-1},C_u} + M_{C_v,C_{v+1}}$
        $NewCost \leftarrow M_{C_{u-1},C_v} + M_{C_u,C_{v+1}}$
        **if** $NewCost < CurrentCost$ **then**
            **Inversion:** Flip the sub-route $C[u \dots v]$
            Update sequence $C$ and $Improved \leftarrow$ True
        **end if**
    **end for**
**end while**
**return** $C$

**Algorithm 3** LS-2 (Kombi Pattern Reduction)

**Require:** Object length $L$, item lengths $l_i$, initial pattern set $P$ with frequencies $x$, $MaxFails = 50$
**Ensure:** Reduced pattern set $P$ with minimized setup cost

$FailCount \leftarrow 0$
**while** $FailCount < MaxFails$ **and** $|P| \geq 2$ **do**
  $Success \leftarrow$ False
  ▷ **Step 1: Try 3-to-2 Combination**
  **if** $|P| \geq 3$ **then**
    Randomly select 3 patterns $p_1, p_2, p_3 \in P$ with frequencies $x_1, x_2, x_3$
    Compute target production $s_i = a_{1i}x_1 + a_{2i}x_2 + a_{3i}x_3, \forall i \in I$
    Define possible new frequency pairs: $(x^*, x^{**}) \in \{(x_1, x_2{+}x_3), (x_2, x_1{+}x_3), (x_3, x_1{+}x_2)\}$
    Search for 2 valid patterns $a^*, a^{**}$ such that $\sum a_i^* l_i \leq L$ **and** $\sum a_i^{**} l_i \leq L$
    subject to exact production: $a_i^* x^* + a_i^{**} x^{**} = s_i, \forall i \in I$
    **if** valid patterns $a^*, a^{**}$ are found **then**
      Remove $p_1, p_2, p_3$ from $P$
      Add $(a^*, x^*)$ and $(a^{**}, x^{**})$ to $P$
      $Success \leftarrow$ True
    **end if**
  **end if**
  ▷ **Step 2: Try 2-to-1 Combination**
  **if not** $Success$ **then**
    Randomly select 2 patterns $p_1, p_2 \in P$ with frequencies $x_1, x_2$
    $x_{new} \leftarrow x_1 + x_2$
    Compute target production $s_i = a_{1i}x_1 + a_{2i}x_2, \forall i \in I$
    **if** $s_i \pmod{x_{new}} == 0, \forall i \in I$ **then**
      $a_i^{new} \leftarrow s_i / x_{new}$
      **if** $\sum a_i^{new} l_i \leq L$ **then**
        Remove $p_1, p_2$ from $P$
        Add $(a^{new}, x_{new})$ to $P$
        $Success \leftarrow$ True
      **end if**
    **end if**
  **end if**
  ▷ **Counters Update**
  **if** $Success$ **then**
    $FailCount \leftarrow 0$ ▷ Restart fail counter upon improvement
  **else**
    $FailCount \leftarrow FailCount + 1$
  **end if**
**end while**
**return** $P$